

Spec-Driven Development (SDD)

KI-assistierte Softwareentwicklung

Handreichung für Studium und Praxis

Worum es geht.

Diese Handreichung erklärt, wie sich Softwareentwicklung durch KI-Coding-Agenten verändert. Sie grenzt das fehleranfällige „Vibe Coding“ vom strukturierten Spec-Driven Development ab, ordnet SDD in bekannte Verfahren des Software-Engineerings ein, benennt seine Prinzipien und Grenzen und liefert ein durchgängiges Beispiel, einen praxistauglichen Workflow sowie einen Meta-Prompt zum schnellen Erstellen von Spezifikationen. Der zweite Teil ist ein Tutorial, das den Weg von der leeren Projektmappe bis zum laufenden Prototyp Schritt für Schritt zeigt.

Inhalt

Teil I — Die Methode	2
1. Was ist SDD — und woher es kommt	2
2. Das Problem: „Vibe Coding“	2
3. Die Prinzipien des SDD	3
4. Das Spezifikationsdokument	3
5. Der Workflow in der Praxis	7
6. Werkzeuglandschaft	9
7. Grenzen — und wann SDD sich nicht lohnt	9
8. Spezifikationen schnell erstellen: der Meta-Prompt	10
9. Glossar	11
Teil II — In der Praxis: Tutorial am Beispiel Predictive Maintenance	12
Bevor du loslegst: iterativ statt Wasserfall	12
0. Voraussetzungen & Setup	13
1. Das Fundament: die Spec ins Projekt legen	14
2. Spec prüfen und Plan erzeugen — noch ohne Code	15
3. In kleine Aufgaben zerlegen	16
4. Implementieren — Tests zuerst	16
5. Verifizieren & weiterentwickeln	17
6. Spickzettel: Continue vs. Claude Code	18
7. Häufige Stolpersteine	18
Quellen & weiterführende Links	18

1. Was ist SDD — und woher es kommt

Spec-Driven Development stellt eine maschinenlesbare Spezifikation ins Zentrum des Entwicklungsprozesses. Zuerst wird beschrieben, *was* ein System leisten soll und unter welchen Bedingungen; aus dieser Beschreibung leitet ein KI-Agent Plan, Tests und Code ab. Die Spezifikation ist der verbindliche Vertrag — die „Source of Truth“. Code und Tests sind nachgelagerte Artefakte, die sich an ihr messen lassen müssen.

Der entscheidende Effekt: Der menschliche Aufwand verschiebt sich von der Implementierung hin zur präzisen Beschreibung von Anforderungen, Architektur und Akzeptanzkriterien — also dorthin, wo Fehler am billigsten zu beheben sind.

Vertiefung — Einordnung

SDD ist keine Erfindung von 2025, sondern eine Neuverpackung bewährter Ideen des Software-Engineerings:

- **Contract-First-Design** — die Schnittstelle (z. B. als OpenAPI/Swagger-Dokument) steht vor der Implementierung fest.
- **Design by Contract** (Bertrand Meyer, Eiffel) — Vor- und Nachbedingungen als Vertrag zwischen Komponenten.
- **Formale Spezifikation** (Z, VDM) und **Behaviour-Driven Development** (Gherkin/Cucumber) — Verhalten in prüfbarer Form.
- **Architecture Decision Records** — bewusst dokumentierte, nachvollziehbare Entwurfsentscheidungen.

Neu ist allein der Auslöser: Leistungsfähige Coding-Agenten machen es erstmals praktikabel, ausführbaren Code unmittelbar aus einer präzisen Spezifikation zu erzeugen. SDD ist damit weniger ein Bruch als die konsequente Rückkehr zum Requirements Engineering unter neuen Vorzeichen.

2. Das Problem: „Vibe Coding“

Der Begriff „Vibe Coding“ wurde Anfang 2025 geprägt¹. Sein Kern ist nicht bloß „einfacher Prompt, dann iterieren“, sondern dass man den generierten Code akzeptiert, ohne ihn vollständig zu lesen oder zu verstehen — man gibt sich den „Vibes“ hin und vertraut dem Modell. Für schnelle Prototypen, Wegwerf-Skripte und Exploration ist das ein legitimes, oft sehr produktives Vorgehen.

Bei ernsthaften, langlebigen Systemen treten jedoch typische Nachteile auf: Der Agent trifft willkürliche architektonische Entscheidungen, die Ergebnisse sind schwer reproduzierbar, es fehlt die Nachvollziehbarkeit, und der Code „kompiliert zwar, trifft aber die eigentliche Absicht nicht“. Der klassische Planungs- und Reviewzyklus wird übersprungen, was Wartung und Teamarbeit erschwert.

Wichtig — keine Schwarz-Weiß-Frage

Vibe Coding ist nicht per se schlecht; es ist das richtige Werkzeug für Wegwerf-Code und schnelle Experimente. SDD lohnt sich, sobald Korrektheit, Wartbarkeit, Teamarbeit, Sicherheit oder rechtliche Nachvollziehbarkeit zählen. Die Kunst liegt darin, das passende Verfahren bewusst zu wählen.

¹Begriff geprägt von Andrej Karpathy, Februar 2025.

3. Die Prinzipien des SDD

Die folgenden acht Prinzipien beschreiben die Denkweise hinter SDD:

- **Spezifikation als Source of Truth.** Die Spec ist der maßgebliche Vertrag. Bei Abweichungen wird zuerst die Spec geprüft und angepasst, nicht ad hoc der Code.
- **Trennung von „Was“ und „Wie“.** Der Mensch definiert das Was (Geschäftslogik, Anforderungen), der Agent übernimmt das Wie (Syntax, Boilerplate, Implementierungsdetails).
- **Shift-Left der menschlichen Kontrolle.** Das Review findet bei Spezifikation und Architektur statt — bevor Code entsteht, wo Fehler am günstigsten zu korrigieren sind.
- **Leitplanken statt Determinismus.** Enge Vorgaben zu Bibliotheken, Fehlerbehandlung und Namenskonventionen verkleinern den Lösungsraum und reduzieren Halluzinationen. Sie machen die Generierung nicht deterministisch — LLMs bleiben stochastisch —, aber kalkulierbarer.
- **Verifizierbarkeit durch Design.** Eine Spec ist nur so gut, wie sie sich in messbare Akzeptanzkriterien (etwa Unit- und E2E-Tests) übersetzen lässt.
- **Nachvollziehbarkeit (Traceability).** Jedes Code-Artefakt lässt sich auf ein Spec-Element zurückführen. Dies ist der eigentliche Mehrwert gegenüber Vibe Coding und unverzichtbar in regulierten Umfeldern.
- **Lebendes Dokument.** Die Spec wird versioniert, lebt im Repository und wird über Reviews bzw. Pull Requests fortgeschrieben — nicht einmalig geschrieben und beiseitegelegt.
- **Werkzeug-Unabhängigkeit.** SDD ist eine Methodik, kein Produkt. Sie funktioniert mit verschiedenen Agenten und Stacks.

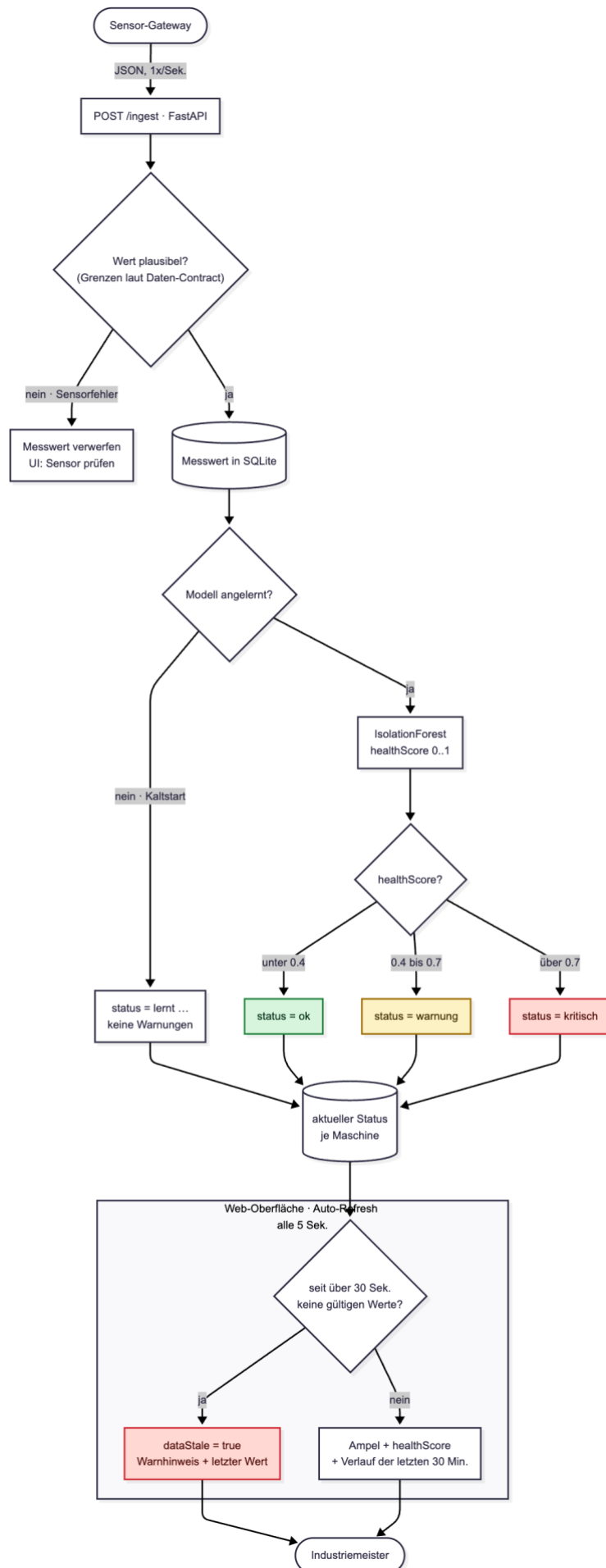
Vertiefung — „Source of Truth“ realistisch betrachtet

Das Ideal lautet: bei Fehlern stets die Spec ändern und den Code neu generieren, nie umgekehrt. In der Praxis ist das eine Zielvorstellung, kein Automatismus. Generierter Code braucht weiterhin Review; eine Spec erfasst selten 100 % des Verhaltens; und das verlustfreie Regenerieren von Code aus der Spec („Round-Tripping“) funktioniert heute nicht zuverlässig. Wer am Code vorbei arbeitet, erzeugt „Spec-Drift“ — Code und Spec laufen auseinander. Behandle das Prinzip daher als Disziplin, die das Team aktiv aufrechterhält.

4. Das Spezifikationsdokument

Eine Spec wird als maschinenlesbarer Vertrag (typischerweise Markdown) in das Projekt gelegt und dem Coding-Agenten als Kontext übergeben. Das folgende Beispiel stammt aus der Arbeitswelt vieler Kursteilnehmer: ein Prototyp für Predictive Maintenance. Eine Maschine wird mit Sensoren nachgerüstet, ein ML-Modell bewertet laufend ihren Zustand, und ein Industriemeister überwacht das Ergebnis über eine einfache Web-Oberfläche.

Das Beispiel ist bewusst nah am Proof of Concept gehalten — denn häufig entsteht zuerst ein Prozess-Prototyp, den später externe Fachleute zum produktiven System ausbauen. Gerade dann ist eine grundsätzliche Anfangs-Spezifikation Gold wert: Sie ist zugleich Bauplan und Übergabedokument. Beachte besonders die Abschnitte 7 und 8 — sie machen Annahmen und Scope explizit und sagen dem Folgeteam genau, wo es ansetzen muss.



Datenfluss des Prototyps: von der Sensorik über Plausibilitätsprüfung und Anomalie-Score bis zur Ampel-Oberfläche.

Beispiel: spec.md (Predictive-Maintenance-Prototyp)

```
# Spezifikation: Predictive-Maintenance-Prototyp für eine Produktionsmaschine

> Status: Prototyp / Proof of Concept · Source of Truth für dieses Projekt.
> Änderungen am Verhalten werden zuerst hier eingetragen, dann im Code umgesetzt.

## 0. Zweck & Reifegrad
PROTOTYP / Proof of Concept. Ziel ist NICHT der produktive Dauerbetrieb, sondern der
Nachweis, dass sich aus Sensordaten ein brauchbares Frühwarnsignal für den Wartungsbedarf
ableiten lässt. Die skalierte, produktive Umsetzung erfolgt anschließend mit externen
Fachleuten (siehe Abschnitt 8). Diese Spec ist zugleich Bauplan und Übergabedokument.

## 1. Daten-Contract (Sensorik)
- Quelle: Sensor-Gateway sendet Messwerte als JSON per HTTP POST /ingest.
- Abtastrate (Prototyp): 1 Messung pro Sekunde je Sensor.
- Felder je Nachricht:
  - machineId (string)
  - timestamp (ISO 8601, UTC)
  - vibration (number, mm/s)
  - temperature (number, °C)
  - current (number, A)
- Plausibilitätsgrenzen – Werte außerhalb gelten als SENSORFEHLER, nicht als Maschinenfehler:
  - temperature: -20 bis 150 vibration: 0 bis 50 current: 0 bis 100

## 2. ML-Aufgabe (bewusst einfach gehalten)
- Aufgabentyp: ANOMALIE-ERKENNUNG (unüberwacht). KEINE Ausfallvorhersage in Zeiteinheiten –
im Prototyp liegen i. d. R. noch keine echten Ausfälle als Trainingslabel vor
("Cold-Start-Problem").
- Vorgehen: Das Modell lernt das Normalverhalten aus einer störungsfreien Referenzphase
und meldet Abweichungen als Anomalie-Score.
- Ausgabe je Maschine, alle 5 Sekunden aktualisiert:
  - healthScore (number, 0..1)
  - status ("ok" | "warnung" | "kritisch")
  Schwellen (Prototyp, später kalibrierbar): < 0.4 = ok, 0.4-0.7 = warnung, > 0.7 = kritisch
  - dataStale (boolean) – true, wenn seit mehr als 30 s keine gültigen Werte kamen

## 3. Web-Oberfläche (für den Industriemeister)
- Eine Seite, Auto-Refresh alle 5 Sekunden.
- Zeigt je Maschine: Ampel (grün/gelb/rot), healthScore, Zeitpunkt der letzten Messung
sowie den Verlauf der letzten 30 Minuten als einfaches Liniendiagramm.
- Bei Status "kritisch" oder dataStale: deutlich sichtbarer Warnhinweis.
- Verständlich ohne Schulung – der Meister muss den Zustand sofort deuten können.

## 4. Verhalten in Sonderfällen
- Datenlücke / Sensorausfall: Status NICHT auf "ok" setzen; dataStale = true und
letzten bekannten Wert mit Zeitstempel anzeigen.
- Sensorfehler (Wert außerhalb der Plausibilitätsgrenzen): Messwert verwerfen, im UI als
"Sensor prüfen" markieren, nicht in den healthScore einrechnen.
- Kaltstart (Modell noch nicht angelernt): Status "lernt ..." anzeigen, noch keine
Warnungen erzeugen.

## 5. Technische Constraints (prototyp-tauglich)
- Sprache: Python.
- ML: scikit-learn, z. B. IsolationForest – KEIN Deep Learning im Prototyp.
- Backend: FastAPI. Datenhaltung: SQLite (eine Datei, kein DB-Server).
- Frontend: eine einfache HTML-Seite, die das Backend per fetch abfragt.
- Alles lokal auf einem Rechner lauffähig, keine Cloud-Abhängigkeit.

## 6. Akzeptanzkriterien (Prototyp gilt als erfolgreich, wenn ...)
1. Bei eingespielten Normaldaten bleibt der Status stabil auf "ok".
2. Eine künstlich erzeugte Auffälligkeit (z. B. steigende Vibration) hebt den Status
innerhalb von 15 s auf "warnung"/"kritisch".
3. Bei abgeschaltetem Sensor zeigt das UI binnen 30 s dataStale an.
4. Der Industriemeister deutet den angezeigten Zustand ohne Erklärung richtig.
```

7. Annahmen (explizit, damit sie überprüfbar bleiben)

- Genau eine Maschine; Mehrmaschinen-Betrieb erst in der Produktivversion.
- Läuft im geschützten internen Netz → keine Auth/Verschlüsselung im PoC.
- Eine ausreichend lange störungsfreie Referenzphase zum Anlernen existiert.
- Datenmengen sind klein genug für SQLite und einen einzelnen Prozess.

8. Bewusst NICHT im Prototyp – Übergabe an Fachleute

- Skalierung auf viele Maschinen, Hochverfügbarkeit, Zeitreihen-Datenbank.
- Echte Restlebensdauer-Vorhersage auf Basis gelabelter Ausfälle.
- Authentifizierung, Rollen, Audit, Alarmierung per E-Mail/SMS.
- Modell-Monitoring, Drift-Erkennung, regelmäßiges Re-Training.
- OT-/Sicherheitsanforderungen (Trennung vom Produktionsnetz, einschlägige Normen).

Vertiefung — was eine gute KI-/ML-Prototyp-Spec ausmacht

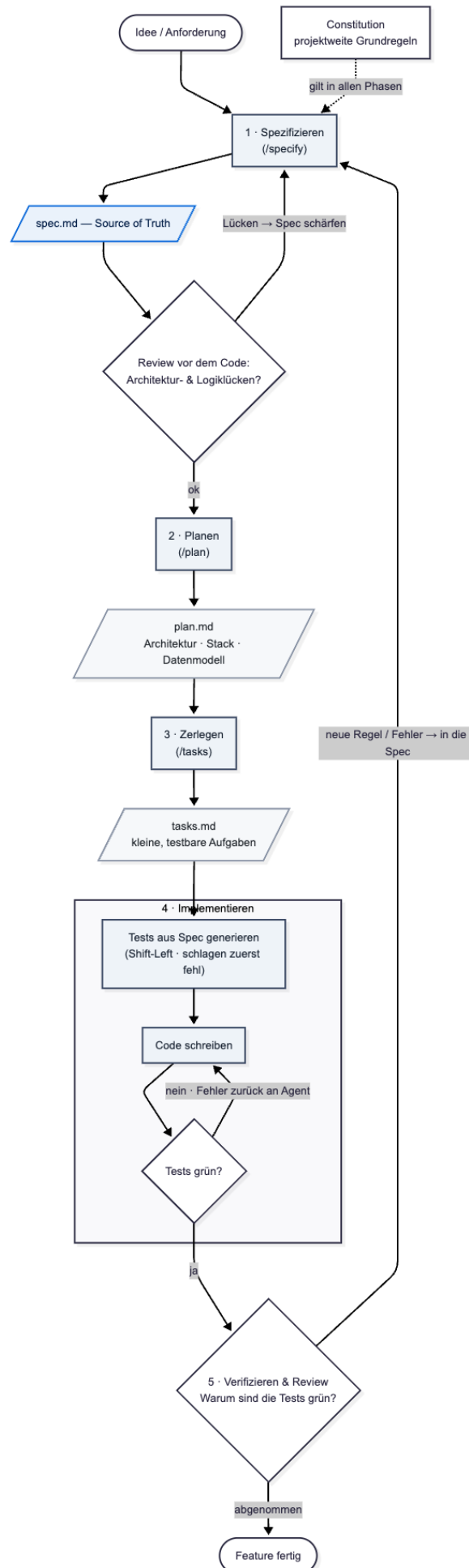
Bei einem ML-Prototyp entscheiden andere Dinge über die Qualität als bei klassischer Software. Die wichtigsten:

- **ML-Aufgabe präzise benennen:** Anomalie-Erkennung ist nicht dasselbe wie Ausfallvorhersage. Wer das offenlässt, bekommt vom Agenten — oder später vom externen Team — womöglich etwas ganz anderes als gemeint.
- **Cold-Start ehrlich behandeln:** Predictive Maintenance bräuchte eigentlich echte Ausfalldaten zum Lernen, die im Prototyp fast nie vorliegen. Die Spec macht aus dieser Not eine bewusste Entscheidung: unüberwachte Anomalie-Erkennung auf Basis des Normalverhaltens.
- **Daten-Contract ist das Fundament:** Format, Abtastrate, Einheiten und Plausibilitätsgrenzen. Ein ML-System ist nur so gut wie die Klarheit über seine Eingangsdaten — „garbage in, garbage out“.
- **Sensorfehler ≠ Maschinenfehler:** Diese Unterscheidung entscheidet, ob das Frühwarnsystem vertrauenswürdig ist. Ein defekter Sensor darf keinen Maschinenalarm auslösen — und umgekehrt.
- **Verständlichkeit für den Nutzer:** Der Industriemeister ist kein Data Scientist. Eine Ampel, die er ohne Schulung deuten kann, ist mehr wert als ein präziser, aber unverständlicher Score.
- **Annahmen und Scope explizit machen:** Gerade weil später Fachleute übernehmen, ist die Spec die Übergabestelle. Die Abschnitte „Annahmen“ und „Bewusst NICHT im Prototyp“ sind kein Beiwerk, sondern der eigentliche Wert — sie verhindern, dass ein Proof of Concept versehentlich für ein fertiges Produkt gehalten wird.

5. Der Workflow in der Praxis

Der heute etablierte Ablauf gliedert sich in fünf Phasen, die jeweils ein eigenes, überprüfbares Artefakt erzeugen, das die nächste Phase speist. Über allen Phasen liegt die **Constitution** — projektweite Grundregeln (im Tutorial die **CLAUDE.md** bzw. **CONVENTIONS.md**). Toolkits wie GitHub Spec Kit bilden den Ablauf über Befehle wie **/specify**, **/plan** und **/tasks** ab; diese Befehle stehen stellvertretend für die jeweilige Phase — die Methode selbst bleibt werkzeugunabhängig.

- **Spezifizieren.** Die Spec schreiben und den Agenten zunächst nur auf Architektur- und Logiklücken prüfen lassen — noch ohne Code.
- **Planen.** Einen technischen Plan ableiten: Architektur, Datenmodell, Schnittstellen, gewählte Bibliotheken und Begründungen.
- **Zerlegen.** Den Plan in kleine, einzeln testbare Aufgaben aufteilen.
- **Implementieren.** Zuerst Tests aus der Spec generieren (Shift-Left; sie schlagen anfangs fehl), dann Code schreiben, bis die Tests grün sind. Fehlermeldungen werden dem Agenten zur Selbstkorrektur zurückgegeben.
- **Verifizieren & Review.** Prüfen, *warum* die Tests grün sind, nicht nur *dass* sie es sind. Neue Regeln wandern in die Spec — niemals als formloser Zuruf an den Code.



Der SDD-Ablauf als Kette aus fünf Phasen mit ihren Artefakten und Rückkopplungen.

Im Diagramm wird der Zusammenhang sichtbar: Aus der Idee entsteht `spec.md` als Source of Truth. Vor der ersten Zeile Code steht ein menschliches Review (Shift-Left); gefundene Lücken fließen zurück in die Spec, nicht in eine spätere Phase. Erst nach Freigabe entstehen `plan.md` und `tasks.md`, dann die Tests und der Code.

Entscheidend sind die **drei Rückkopplungen**: erstens das Review vor dem Code, zweitens die Test-Korrektur-Schleife innerhalb der Implementierung (die nur den Code betrifft, nicht die Anforderungen) und drittens die übergeordnete Schleife, die jede neue Regel und jeden gefundenen Fehler über die Spezifikation führt. Die Pfeile führen deshalb zurück zur Phase „Spezifizieren“, nie direkt zum Code. Erst wenn das Feature abgenommen ist, endet der Durchlauf.

Achtung — Grenzen der autonomen Fehlerbehebung

Ein Agent kann Tests auch „grün“ bekommen, indem er Assertions abschwächt oder Sonderfälle umgeht, statt das Problem zu lösen. Das Review muss daher prüfen, ob die Tests das richtige Verhalten erzwingen — nicht nur, ob sie bestehen. Begrenze zudem die Zahl der Selbstkorrektur-Schleifen, um Endlos-Iterationen zu vermeiden.

6. Werkzeuglandschaft

Den Begriff SDD und einen konkreten Prozess hat vor allem **GitHub Spec Kit** geprägt²: ein quelloffenes, bewusst werkzeug-unabhängiges Toolkit mit einer CLI (`specify`), Vorlagen und den Phasen Spec → Plan → Tasks → Implement. Eine projektweite „Constitution“ hält übergreifende Grundregeln fest. Es arbeitet mit zahlreichen Agenten zusammen.

Coding-Agenten, mit denen SDD praktiziert wird, sind unter anderem Claude Code, GitHub Copilot, Gemini CLI, Cursor und Aider. Daneben existieren weitere spec-orientierte Umgebungen. Entscheidend ist nicht das Produkt, sondern die Disziplin: Spezifikation zuerst, Code als Ableitung.

7. Grenzen — und wann SDD sich nicht lohnt

SDD ist kein Allheilmittel. Eine ehrliche Handreichung benennt die Kehrseiten, damit das Verfahren bewusst eingesetzt wird:

- **Overhead**: Eine gute Spec zu schreiben kostet Zeit. Bei Wegwerf-Code und Prototypen ist das unwirtschaftlich.
- **Fehlbare Spec**: Eine Spezifikation kann genauso unvollständig oder falsch sein wie Code. SDD verlagert das Denken, es ersetzt es nicht.
- **Restrisiko Halluzination**: Auch innerhalb enger Constraints kann ein Modell Fehler produzieren.
- **Spec-Drift & Round-Tripping**: Code und Spec laufen auseinander, wenn man am Code vorbei arbeitet; das verlustfreie Regenerieren ist unreif.
- **Falsches Sicherheitsgefühl**: Eine formal wirkende Spec verleitet dazu, Review und kritisches Denken zu vernachlässigen.

Faustregel zur Auswahl

SDD lohnt sich bei langlebigen, mehrköpfigen, korrektkeits- oder compliance-kritischen Projekten und beim Arbeiten in bestehenden Codebasen. Für Exploration, Lernprojekte und Einweg-Skripte genügt Vibe Coding — und ist oft schneller.

²GitHub Spec Kit, als Open-Source-Toolkit veröffentlicht am 2. September 2025 (MIT-Lizenz).

8. Spezifikationen schnell erstellen: der Meta-Prompt

Statt Spezifikationen mühsam selbst zu schreiben, dreht man den Spieß um und lässt sich von der KI interviewen. Den folgenden Prompt in einen LLM-Chat kopieren und den Platzhalter durch die eigene Idee ersetzen.

Du agierst ab sofort als sehr erfahrener Senior Software Architect und Product Manager. Mein Ziel ist es, nach den Prinzipien des Spec-Driven Development (SDD) zu arbeiten. Am Ende sollst du mir eine wasserdichte, maschinenlesbare Markdown-Spezifikation (spec.md) für ein Feature/Projekt schreiben, die ich direkt an einen KI-Coding-Agenten übergeben kann.

Hier ist meine noch sehr vage Idee:

[HIER DEINE IDEE EINTRAGEN, z. B. "Ich brauche ein internes Dashboard, in dem Mitarbeiter Urlaubsanträge einreichen und Vorgesetzte sie genehmigen."]

WICHTIGE REGEL: Schreibe die Spezifikation noch NICHT.

Bevor wir sie erstellen, musst du alle Unklarheiten beseitigen. Stelle mir eine strukturierte, nummerierte Liste der 10 bis 14 wichtigsten Fragen. Gruppiere sie in folgende Kategorien:

1. Geschäftslogik & Core-Feature-Verhalten
2. Edge Cases (unerwartetes Verhalten, Fehler, Nebenläufigkeit)
3. Security, Authentifizierung & Berechtigungen
4. Technische Constraints (Stack, Datenbank, APIs)
5. Nicht-funktionale Anforderungen (Performance, Skalierung, Logging, Datenschutz)
6. Testing & Akzeptanzkriterien

Stelle präzise, technische Fragen. Biete bei komplexen Punkten direkt 2 bis 3 Best-Practice-Optionen (A, B, C) an, aus denen ich wählen kann.

Warte nach deinen Fragen auf meine Antworten. Wenn meine Antworten weitere Lücken aufwerfen, hake nach. Deine Einschätzung "keine Lücken mehr" ist eine Heuristik, keine Garantie – bleibe daher kritisch. Erst wenn du hinreichend sicher bist, werde ich sagen: "Erstelle jetzt die spec.md".

So geht es nach dem Prompt weiter

- **Beantworten:** kurz und knapp auf die Fragen antworten (z. B. „Zu 3: Option B. Zu 4: React und Firebase“).
- **Nachschärfen:** eventuelle Rückfragen der KI klären.
- **Generieren:** den finalen Befehl geben: „Perfekt. Erstelle nun die finale spec.md, strukturiert nach API-Contract, Validierung, Geschäftslogik, Fehlerbehandlung, Constraints und Akzeptanzkriterien.“

9. Glossar

Spec-Driven Development (SDD) — Methodik, die eine maschinenlesbare Spezifikation ins Zentrum stellt; Code wird daraus abgeleitet.

Vibe Coding — Generierten Code akzeptieren und über Folge-Prompts iterieren, ohne ihn vollständig zu durchdringen.

Source of Truth — Die Spezifikation als einzige verbindliche Referenz für das Soll-Verhalten.

Shift-Left — Qualitätssicherung früh im Prozess verankern — hier: Review bei Spec und Architektur statt erst am Code.

Constraint (Leitplanke) — Verbindliche Vorgabe (Stack, Bibliotheken, Konventionen), die den Lösungsraum des Agenten einengt.

Traceability — Nachvollziehbarkeit: jedes Code-Artefakt lässt sich auf ein Spec-Element zurückführen.

Akzeptanzkriterium — Messbare, prüfbare Bedingung, an der sich die Erfüllung einer Anforderung feststellen lässt.

Cold-Start-Problem (ML) — Schwierigkeit, ein Vorhersagemodell zu trainieren, wenn (noch) keine Beispiele für den seltenen Zielfall — etwa Maschinenausfälle — vorliegen.

Anomalie-Erkennung — Unüberwachtes ML-Verfahren, das aus dem Normalverhalten lernt und Abweichungen meldet — geeignet, wenn keine gelabelten Fehlerfälle vorliegen.

Constitution — In manchen Toolkits projektweite Grundregeln, die über einzelnen Specs stehen.

Round-Tripping — Idee, Code jederzeit verlustfrei aus der Spec neu zu generieren — aktuell technisch unreif.

Spec-Drift — Auseinanderlaufen von Spezifikation und Code, wenn am Code vorbei gearbeitet wird.

Teil II — In der Praxis: Tutorial am Beispiel Predictive Maintenance

Bevor du loslegst: iterativ statt Wasserfall

Vielleicht denkst du beim Blick auf das folgende Tutorial: „*Wie soll ich solche Dokumente erstellen, bevor ich überhaupt etwas programmiert habe?*“ Das Tutorial zeigt einen idealen, vollständigen Durchlauf — das Ziel, nicht den Stand, mit dem du beginnst. Niemand schreibt vor dem ersten Tastendruck eine perfekte, fertige Spezifikation; genau das wäre nicht SDD, sondern Wasserfall. In Wirklichkeit wächst die Spec mit dem Projekt: Du startest mit einem Skelett und füllst es Schritt für Schritt. Die saubere Endfassung ist das Ergebnis mehrerer Runden, nicht ihre Voraussetzung.

Der häufigste Irrtum ist, SDD als „alles vorab durchdenken“ zu lesen. SDD ist das Gegenteil — die Spezifikation ist ein lebendes Dokument. Im Prozessdiagramm führt jede Korrektur zurück in die Spec; das ist bereits Iteration.

„Altes Software-Dev“ (Wasserfall)	So ist SDD gemeint (iterativ)
Erst das komplette Dokument, dann Code	Dünne Spec, kurzer Durchlauf, dann nachschärfen
Die Spec ist fertig und unveränderlich	Die Spec ist ein lebendes Dokument
Vollständigkeit vor dem ersten Schritt	Nur so viel wie für den nächsten Schritt nötig
Ein großer Wurf	Viele kleine, vollständige Schnitte

Merksatz — Just enough spec

So viel Spezifikation, wie für den nächsten kleinen Schritt nötig ist, nicht mehr. Ein SDD-Durchlauf (Spec → Plan → Tasks → Implement → Verify) ist kurz und wird wiederholt — im Grunde ein Sprint. Die Iterationseinheit ist die Spec, nicht der Code.

Faustregel für den Einstieg — Stabiles früh, Volatiles spät

Vorab gehört, was billig zu denken und teuer zu ändern ist. Wachsen darf, was man ohnehin erst später genauer weiß.

Früh festlegen (am Beispiel Predictive Maintenance): der ML-Aufgabentyp (Anomalie-Erkennung statt Ausfallvorhersage), der Daten-Contract und ein bis zwei Akzeptanzkriterien.

Später wachsen lassen: konkrete Schwellenwerte, Feinheiten und Sonderfälle.

So fängst du konkret an

1. Mit einem Skelett starten: Zweck, Daten-Contract und eine einzige Akzeptanzbedingung genügen.
2. Den ersten kleinen Schnitt bauen — ein vollständiger, kurzer Durchlauf wie unten im Tutorial.
3. Den laufenden Prototyp „fragen“, was als Nächstes in die Spec muss — und die Spec ergänzen.

Worum es im Tutorial geht.

Es zeigt ganz konkret, wie man ein KI-Projekt nach der SDD-Methode beginnt — von der leeren Projektmappe bis zum laufenden Prototyp. Als Beispiel dient die Predictive-Maintenance-Spezifikation aus Teil I. Jeder Schritt ist für beide Werkzeuge beschrieben: die Continue-Extension in VS Code und Claude Code.

Roter Faden: Spec → Plan → Tasks → Implementieren → Verifizieren. Der Code entsteht zuletzt — und immer aus der Spec.

0. Voraussetzungen & Setup

Du brauchst vorab drei Dinge: VS Code, Python und ein KI-Werkzeug. Lege dann das Projekt an und installiere die Bibliotheken aus der Spec.

Werkzeug einrichten — eine der beiden Varianten

So geht's in Continue (VS Code)

- In VS Code links auf „Extensions“ gehen, nach „Continue“ suchen, installieren. Danach erscheint links das Continue-Symbol — darüber öffnet sich das Seitenpanel.
- Beim ersten Start anmelden bzw. ein Modell hinterlegen.
- Continue kennt vier Funktionen: **Autocomplete**, **Chat**, **Edit** und **Agent**. Die Betriebsart wählst du im Auswahlfeld unter dem Eingabefeld; mit Strg/Cmd + . schaltest du durch.

So geht's in Claude Code

Claude Code installieren (aktuelle Schritte siehe Doku-Link am Ende). Empfohlen ist der native Installer; alternativ per npm (benötigt Node.js 18+):

```
# Variante A (empfohlen):
curl -fsSL https://claude.ai/install.sh | bash

# Variante B (npm):
npm install -g @anthropic-ai/claude-code

# danach im Projektordner starten und beim ersten Mal anmelden:
claude
```

Claude Code braucht einen kostenpflichtigen Zugang (Pro/Max) oder einen API-Schlüssel. Die VS-Code-Erweiterung „Claude Code“ zeigt Änderungen als Diff direkt im Editor an.

Kostenhinweis

Agent-Modus und Claude Code verbrauchen Tokens (= Geld). Fang mit kleinen Aufgaben an und behalte den Verbrauch im Blick. Wer sparen will, nutzt Continue mit einem günstigen oder lokalen Modell für die einfachen Schritte.

1. Das Fundament: die Spec ins Projekt legen

Die Spezifikation ist die „Source of Truth“. Sie gehört als Datei ins Projekt — nicht in den Chatverlauf, denn der ist flüchtig. Lege im Projektordner eine Datei `spec.md` an und füge die Predictive-Maintenance-Spezifikation aus Teil I (Abschnitt 4) ein.

Daneben braucht das Werkzeug dauerhafte Projektregeln (die „Constitution“): bevorzugte Sprache, Konventionen, der Hinweis, dass `spec.md` verbindlich ist. So musst du dich nicht in jeder Sitzung wiederholen. Beide Werkzeuge nutzen dasselbe Prinzip — nur Dateiname und Einbindung unterscheiden sich:

So geht's in Claude Code

Lege eine Datei **CLAUDE.md** im Projekt-Stammverzeichnis an. Claude Code liest sie zu Beginn jeder Sitzung automatisch.

So geht's in Continue (VS Code)

Lege denselben Inhalt als **CONVENTIONS.md** im Projekt-Stammverzeichnis an und hänge ihn im Chat mit **@CONVENTIONS.md** an (alternativ als Continue-Regel hinterlegen).

Eine ausgereifte Projektregel-Datei sieht etwa so aus:

```
# Projektregeln – Predictive-Maintenance-Prototyp

> Diese Datei ist das "Grundgesetz" des Projekts (Constitution).
> Claude Code liest sie zu Beginn jeder Sitzung automatisch.
> Für die Continue-Extension: denselben Inhalt als CONVENTIONS.md ablegen
> und im Chat per @CONVENTIONS.md anhängen (oder als Continue-Regel hinterlegen).

## Worum es geht
Prototyp / Proof of Concept für Predictive Maintenance:
Sensordaten → Anomalie-Erkennung → Ampel-Status → einfache Web-Oberfläche
für den Industriemeister.

## Wichtigste Regel (Spec-Driven Development)
- spec.md ist die Source of Truth. Verhalten, Constraints und Akzeptanzkriterien stehen dort.
- Neue Anforderungen oder Fehler kommen ZUERST in spec.md, danach in den Code – niemals umgekehrt.

## Tech-Stack & Constraints (aus spec.md, Abschnitt 5)
- Sprache: Python.
- ML: scikit-learn (IsolationForest) – kein Deep Learning.
- Backend: FastAPI. Datenhaltung: SQLite (eine Datei, kein DB-Server).
- Frontend: eine einfache HTML-Seite, die das Backend per fetch abfragt.
- Lokal lauffähig, keine Cloud-Abhängigkeit.
- Keine zusätzlichen Bibliotheken einführen, die nicht in spec.md genannt sind; vorher fragen.

## Arbeitsweise, die ich erwarte
- Bei größeren Schritten zuerst kurz den Plan erläutern, dann umsetzen.
- Tests zuerst: zu den Akzeptanzkriterien (spec.md, Abschnitt 6) pytest-Tests
  schreiben, die zunächst fehlschlagen – danach den Code, bis sie grün sind.
- In kleinen, einzeln testbaren Schritten entlang tasks.md arbeiten.
- Tests selbst ausführen und das Ergebnis zeigen. Änderungen als nachvollziehbare Diffs halten.

## Code-Stil
- Einfach und gut lesbar; Kommentare auf Deutsch sind willkommen.
- Type Hints verwenden; kleine, fokussierte Funktionen und Module.

## Nicht im Prototyp (siehe spec.md, Abschnitt 8)
- Keine Authentifizierung, kein Mehrmaschinen-Betrieb, kein Modell-Monitoring.
- Im Zweifel: lieber einfach halten und in spec.md vermerken.
```

Tipp — von Anfang an versionieren

Initialisiere Git und committe nach jeder Phase. So hast du immer einen Rückfallpunkt:

```
git init
git add spec.md CLAUDE.md
git commit -m "chore: Spezifikation und Projektregeln"
```

2. Spec prüfen und Plan erzeugen — noch ohne Code

Das ist der „Shift-Left“-Moment: Bevor eine Zeile Code entsteht, lässt du das Modell die Spezifikation auf Lücken prüfen. Arbeite dafür in einem Modus, der nur liest und nichts verändert.

Prompt-Beispiel

Lies @spec.md und @CLAUDE.md. Du bist erfahrener Senior-Entwickler. Prüfe die Spezifikation auf Architektur- und Logiklücken, BEVOR wir Code schreiben. Stelle mir gezielte Rückfragen zu allem, was unklar oder widersprüchlich ist. Schreibe noch keinen Code.

So geht's in Continue (VS Code)

- Neuen Chat öffnen. Im Auswahlfeld den Plan-Modus wählen (liest nur, ändert nichts).
- Dateien als Kontext anhängen mit **@spec.md** und **@CLAUDE.md**. Den Prompt oben einfügen und absenden.

So geht's in Claude Code

- Im Projektordner **claude** starten. In den Plan-Modus wechseln (sicheres, nur lesendes Arbeiten).
- Dateien mit **@spec.md** in den Prompt holen (Dateien sind nicht automatisch im Kontext). Prompt oben verwenden.

Beantworte die Rückfragen knapp. Tauchen dabei echte Lücken auf, ergänze sie in **spec.md** — nicht im Chat. Danach lässt du den Plan erstellen:

Prompt-Beispiel

Erstelle aus @spec.md einen technischen Plan und speichere ihn als plan.md: Projektstruktur (Dateien/Module), Datenfluss, gewählte Bibliotheken und kurze Begründungen. Halte dich an die Constraints in spec.md. Noch kein Implementierungscode.

Lies **plan.md** durch. Stimmt etwas nicht, korrigierst du die Spec und lässt den Plan neu ableiten — das ist die SDD-Schleife in Aktion.

So sieht die fertige Projektmappe ungefähr aus

```
pdm-prototyp/
spec.md          # Source of Truth
plan.md tasks.md # abgeleitete Artefakte
CLAUDE.md        # Projektregeln (bzw. CONVENTIONS.md für Continue)
main.py          # FastAPI: /ingest, Status, Web-Seite
model.py         # Anlernen + Anomalie-Score (IsolationForest)
simulate_sensor.py # Sensor-Simulator (--anomalie)
static/index.html # Ampel-Oberfläche (Auto-Refresh)
tests/           # pytest
```

Die genaue Struktur bestimmt der Agent — wichtig ist nur, dass sie zu **plan.md** passt.

3. In kleine Aufgaben zerlegen

Aus dem Plan wird eine Aufgabenliste. Kleine, einzeln testbare Schritte sind leichter zu prüfen und gehen seltener schief als ein einziger „Mach mal alles“-Auftrag.

Prompt-Beispiel

*Leite aus @plan.md eine Aufgabenliste ab und speichere sie als tasks.md:
kleine, einzeln testbare Schritte in sinnvoller Reihenfolge, jede mit einem Satz Ziel.*

Eine typische Aufgabenliste für diesen Prototyp sieht etwa so aus:

- Datenmodell und Endpunkt **POST /ingest** (FastAPI) anlegen.
- Plausibilitätsprüfung der Messwerte (Sensorfehler erkennen).
- Messwerte in SQLite speichern.
- Modell aus einer Referenzphase anlernen (IsolationForest).
- **healthScore** berechnen und in Status (ok/warnung/kritisch) übersetzen.
- **dataStale**-Logik (seit mehr als 30 s keine Werte).
- Web-Oberfläche mit Ampel und Auto-Refresh.
- Sensor-Simulator zum Testen ohne echte Hardware.
- Tests zu den Akzeptanzkriterien aus **spec.md**.

4. Implementieren — Tests zuerst

Jetzt darf der Agent Dateien anlegen und Befehle ausführen. Die SDD-Reihenfolge lautet: erst die Tests aus den Akzeptanzkriterien, dann der Code, der sie grün macht.

So geht's in Continue (VS Code)

In den **Agent-Modus** wechseln. Jetzt kann Continue Dateien schreiben und das Terminal nutzen.
Wichtig: Jede vorgeschlagene Änderung als Diff prüfen und erst dann übernehmen.

So geht's in Claude Code

Im normalen Modus arbeiten — Claude Code fragt vor Datei- und Terminal-Aktionen nach.
Änderungen erscheinen als Diff (VS-Code-Erweiterung). Bestätigen oder ablehnen.

Schritt 1 — Tests, die zunächst fehlschlagen:

Prompt-Beispiel

Wir arbeiten @tasks.md Schritt für Schritt ab. Beginne mit den Tests. Schreibe auf Basis der Akzeptanzkriterien in @spec.md (Abschnitt 6) pytest-Tests, die das Verhalten prüfen und zunächst fehlschlagen. Erkläre kurz, was jeder Test prüft. Schreibe noch keinen Implementierungscode.

Schritt 2 — Code, bis die Tests grün sind:

Prompt-Beispiel

Implementiere jetzt den Code, bis alle Tests grün sind. Halte dich strikt an die Constraints in @spec.md Abschnitt 5 (Python, FastAPI, scikit-learn IsolationForest, SQLite). Führe die Tests aus und zeig mir das Ergebnis. Schlagen Tests fehl, korrigiere und wiederhole.

Der Agent führt nun selbst die Schleife „Test ausführen → Fehler lesen → Code anpassen“ aus. Lass ihn iterieren, aber sieh dir die Diffs an.

Schritt 3 — testen ohne echte Maschine:

Prompt-Beispiel

Erzeuge ein Skript simulate_sensor.py, das jede Sekunde plausible Messwerte an /ingest sendet. Mit dem Schalter --anomalie steigt die Vibration langsam an, sodass der Status auf warnung/kritisch wechselt. So kann ich die Akzeptanzkriterien von Hand prüfen.

Prototyp starten und im Browser ansehen — die wichtigsten Befehle stehen im Abschnitt „Nützliche Befehle“ der **CLAUDE.md** (Schritt 1).

5. Verifizieren & weiterentwickeln

Grüne Tests sind notwendig, aber nicht alles. Prüfe, ob sie das richtige Verhalten erzwingen — nicht nur, dass sie bestehen. Gleiche das laufende Verhalten gegen die Akzeptanzkriterien aus **spec.md** ab.

Die wichtigste Gewohnheit

Neue Wünsche oder gefundene Fehler kommen ZUERST in **spec.md** — nie als formloser Zuruf an den Code. Erst danach lässt du den Agenten anpassen. Sonst entsteht „Spec-Drift“: Code und Spezifikation laufen auseinander.

Beispiel: Der Industriemeister möchte zusätzlich die Maschinen-ID im Titel sehen. Du ergänzt das in **spec.md** (Abschnitt 3) und promptest dann:

Prompt-Beispiel

*@spec.md wurde geändert (Abschnitt 3: Maschinen-ID im Titel der Web-Oberfläche).
Passe Implementierung UND Tests entsprechend an und führe die Tests aus.*

6. Spickzettel: Continue vs. Claude Code

Aufgabe	Continue (VS Code)	Claude Code
Datei als Kontext geben	@dateiname im Chat	@dateiname oder /add
Nur lesen / planen (sicher)	Plan-Modus wählen	Plan-Modus (Shift+Tab)
Dateien anlegen, Befehle ausführen	Agent-Modus	Standardmodus (fragt nach)
Projektregeln dauerhaft	CONVENTIONS.md + @-Verweis	CLAUDE.md im Stammverzeichnis
Neue, leere Sitzung	Strg/Cmd + L	/clear
Änderungen prüfen	Diff ansehen, dann „Accept“	In-Editor-Diff, bestätigen

7. Häufige Stolpersteine

- **Blind „Accept“ klicken.** Der Agent ist nicht unfehlbar. Diffs immer lesen, bevor du sie übernimmst.
- **Grün ≠ richtig.** Tests können bestehen, weil sie abgeschwächt wurden. Gegen die Akzeptanzkriterien der Spec gegenprüfen.
- **Kontext überladen.** Nur die relevanten Dateien anhängen; bei Themenwechsel eine neue Sitzung starten. Das spart Tokens und verbessert die Antworten.
- **Sensible Daten in die Cloud.** Keine echten Betriebs- oder Maschinendaten an Cloud-Modelle geben, solange das nicht freigegeben ist. Für den Prototyp reichen simulierte Daten.
- **Prototyp mit Produkt verwechseln.** Abschnitt 8 der Spec sagt, was bewusst offenbleibt. Genau dort setzen später die externen Fachleute an.

Quellen & weiterführende Links

- Claude Code — Dokumentation: <https://docs.claude.com/en/docs/claude-code/overview>
- Continue — Dokumentation: <https://docs.continue.dev>
- GitHub Spec Kit (optionales SDD-Toolkit mit /specify, /plan, /tasks): <https://github.com/github/spec-kit>